

Kernel Threads and User Threads

Last Updated: 2022-02-07

A kernel thread is the schedulable entity, which means that the system scheduler handles kernel threads.

These threads, known by the system scheduler, are strongly implementation-dependent. To facilitate the writing of portable programs, libraries provide *user* threads.

A *kernel thread* is a kernel entity, like processes and interrupt handlers; it is the entity handled by the system scheduler. A kernel thread runs within a process, but can be referenced by any other thread in the system. The programmer has no direct control over these threads, unless you are writing kernel extensions or device drivers. For more information about kernel programming, see *Kernel Extensions and Device Support Programming Concepts*.

A *user thread* is an entity used by programmers to handle multiple flows of controls within a program. The API for handling user threads is provided by the *threads library*. A user thread only exists within a process; a user thread in process *A* cannot reference a user thread in process *B*. The library uses a proprietary interface to handle kernel threads for executing user threads. The user threads API, unlike the kernel threads interface, is part of a POSIX-standards compliant portable-programming model. Thus, a multithreaded program developed on an AIX® system can easily be ported to other systems.

On other systems, user threads are simply called *threads*, and *lightweight process* refers to kernel threads.

Parent topic:

→ [Understanding threads and processes](#)